Лабораторная работа № 16–17
Цикл с предусловием while в C#

Цель работы: научиться повторять действия, пока сохраняется условие, когда число повторений заранее неизвестно и тело может не потребоваться ни разу.

Общие упражнения

1. Проверьте чтение до маркера для «0», «5 → 0», «5 → −2 → 0».

Вход	Ожидаемый результат
0	Количество: 0; сумма: 0
5 → 0	Количество: 1; сумма: 5
5 → −2 → 0	Количество: 2; сумма: 3

2. Проверьте пример буфера для N = 1, 7, 8; результаты (1;0), (8;3), (8;3).

N	Ожидаемый результат (capacity; steps)	Пояснение
1	1; 0	capacity = 1 уже ≥ 1, тело не нужно
7	8; 3	1→2→4→8, три удвоения
8	8; 3	1→2→4→8, условие 8 < 8 ложно

3. Проверьте поиск для пустого массива, {−1}, {2,3}, {2,−1}; объясните число проходов.

Массив	Ожидаемый результат	Число проходов тела
{} (пустой)	«Не найдено»	0
{−1}	индекс 0	0 (условие ложно сразу)
{2, 3}	«Не найдено»	2
{2, −1}	индекс 1	1

4. Объясните, почему печать от 1 до заранее известного N сама по себе не показывает преимущество while.

Если N известно заранее, число проходов фиксировано, и for выражает это намерение явнее (инициализация, условие, шаг в одной строке). while удобен, когда количество проходов определяется данными (маркер, поиск, достижение порога), а не константой.

5. Нарисуйте блок-схему с возможностью нуля проходов.

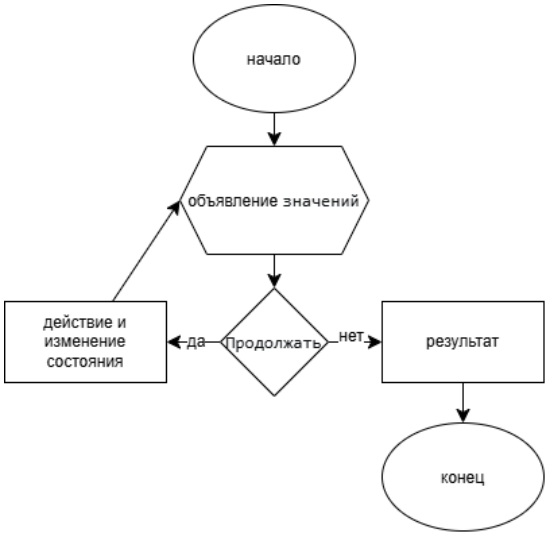


Рисунок 1 - Блок схема

Вариант 6

А. Через while, пока N > 0, удаляйте последнюю цифру положительного int и накапливайте сумму цифр.

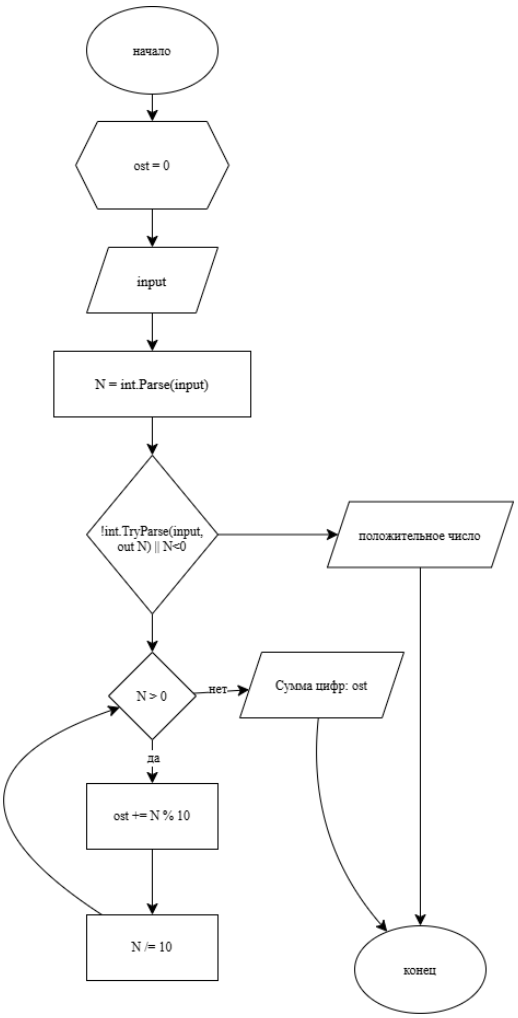


Рисунок 2 - Блок схема

```
int sum = 0;
Console.Write("Введите число: ");
string input = Console.ReadLine();
if (!int.TryParse(input, out int N) || N<0)
{
   Console.WriteLine("положительное число");
   return;
}
while (N > 0)
{
   sum += N % 10;
   N /= 10;
}
Console.Write($"Сумма цифр: {sum}");
```



Рисунок 5 - Результат работы 1

Номер прохода	N до тела	sum до тела	Условие N > 0	N % 10	N после (N / 10)	sum после
0	405	0	—	—	—	—
1	405	0	true	5	40	5
2	40	5	true	0	4	5
3	4	5	true	4	0	9
4	0	9	false	—	—	—

№	Вход	Ожидаемый результат	Фактический
1 (обычный)	12345	15	15
2 (граничный)	1	1	1
3 (особый)	1000000	1	1

Б. Сервис многократно сокращает положительный номер удалением последней цифры, пока не останется одна цифра. Выведите оставшуюся цифру и число удалений.

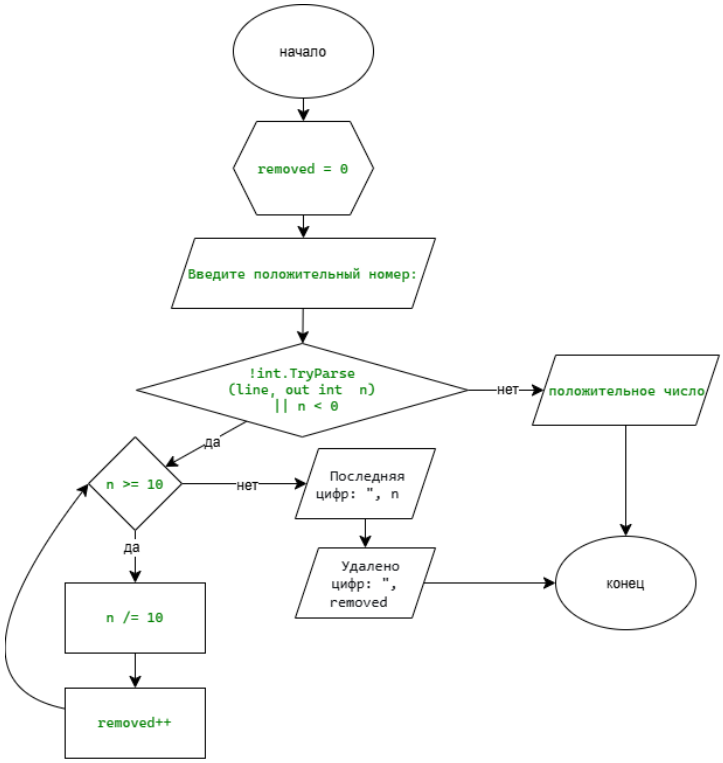


Рисунок 4 - Блок схема

```
int removed = 0;
Console.Write("Введите положительный номер: ");
string line = Console.ReadLine();
if (!int.TryParse(line, out int  n) || n < 0)
{
   Console.WriteLine("положительное число");
   return;
}
while (n >= 10)
{
   n /= 10;
   removed++;
}
Console.WriteLine($"Последняя цифр: {n}");
Console.WriteLine($"Удалено цифр: {removed}");
```

```
Введите положительный номер: 98765
Последняя цифр: 9
Удалено цифр: 4
```

```
Введите положительный номер: 7
Последняя цифр: 7
Удалено цифр: 0
```

Рисунок 5 - Результат работы 2

Проход	n до	removed до	Условие n >= 10	n после	removed после
0	98765	0	—	—	—
1	98765	0	true	9876	1
2	9876	1	true	987	2
3	987	2	true	98	3
4	98	3	true	9	4
5	9	4	false	—	—

Ввод	n >= 10 (1-й проход)	Выполнений цикла	Оставшаяся цифра	Удалено
7	false	0	7	0
10	true	1	1	1
99	true	1	9	1

Контрольные вопросы

1. Когда while выполняется ноль раз?
Ответ: Когда условие в скобках изначально ложно (например, n = 0 при условии n > 0). Программа просто пропускает тело цикла.
2. Почему маркер читают до тела и не включают в результат?
Ответ: Чтобы правильно обработать ситуацию, если маркер введут сразу (тогда цикл не должен запускаться). А не включаем, потому что маркер — это сигнал остановки, а не полезные данные.
3. Какая величина меняется на каждом проходе?
Ответ: Та, которая приближает цикл к завершению. В моих задачах это n (уменьшается делением на 10) и removed (увеличивается на 1).
4. Почему максимум нельзя всегда начинать с нуля?
Ответ: Потому что если все числа отрицательные, ноль окажется больше них, и программа выдаст неверный максимум.
5. Как && защищает доступ к массиву?
Ответ: Он работает слева направо. Если левая часть (проверка индекса) ложна, правая часть (обращение к элементу) даже не выполняется, и программа не падает с ошибкой выхода за границы.
6. Как проверить пустую последовательность?
Ответ: Нужно использовать флаг (например, bool hasData = false) или счетчик (count == 0). Если элементов не было, выводим сообщение об отсутствии данных, а не ноль.
7. Почему сравнение double на точное равенство ненадёжно?
Ответ: Из-за особенностей хранения дробных чисел в памяти компьютера (погрешности округления). Значения могут не совпасть на сотые доли. Нужно использовать Math.Abs(a - b) < eps.
8. Зачем ограничивать число шагов приближения?
Ответ: Для страховки. Если алгоритм не сможет достичь нужной точности, цикл без ограничения шагов превратится в бесконечный и зависнет.
Вывод: выполнила лабораторную работу, изучив цикл с предусловием while. Я поняла, что он отлично подходит, когда количество повторений заранее неизвестно и зависит от самих данных